

SD controller

Xilinx Wiki

Exported on 01/25/2025

Table of Contents

1	Introduction	4
1.1	Zynq	4
1.2	Zynqmp/Versal	4
2	HW/IP features.....	5
2.1	Zynq	5
2.2	ZynqMP/Versal	5
2.3	Features supported by driver	6
2.3.1	Zynq	6
2.3.2	ZynqMP	7
2.3.3	Versal.....	7
3	Missing features, known Issues, limitations	8
4	Kernel configurations	9
5	Devicetree.....	11
6	Performance.....	12
7	Test Procedure.....	13
7.1	Expected output	17
8	Mainline Status	19
9	Change Log	20
10	Related Links.....	22

1 Introduction

1.1 Zynq

1.2 Zynqmp/Versal

The SD/SDIO controller is compatible with the standard SD Host Controller Specification Version 2.0 Part A2 with SDMA (single operation DMA), ADMA1 (4 KB boundary limited DMA), and ADMA2 (ADMA2 allows data of any location and any size to be transferred in a 32-bit system memory - scatter-gather DMA) support. The core also supports up to seven functions in SD1, SD4, but does not support SPI mode. It does support SD high-speed (SDHS) and SD High Capacity (SDHC) card standards. The “Arasan SD3.0 / SDIO3.0 / eMMC4.51 Host Controller”(3MCR Host Controller) is a Host Controller with a AHB/AXI/OCP processor interface. This product conforms to SD Host Controller Standard Specification Version 3.00. The 3MCR Host Controller handles SDIO/SD Protocol at transmission level, packing data, adding cyclic redundancy check (CRC), Start/End bit, and checking for transaction format correctness. The 3MCR Host Controller provides Programmed IO method and DMA data transfer method. In programmed IO method, the Host processor transfers data using the Buffer Data Port Register. Host controller support for DMA can be determined by checking the DMA support in the Capabilities register. DMA allows a peripheral to read or write memory without the intervention from the CPU. The 3MCR Host Controller's Host Controller system address register points to the first data address, and data is then accessed sequentially from that address.

2 HW/IP features

2.1 Zynq

Host mode controller

- Four I/O signals (MIO or EMIO)
- Command, Clock, CD, WP, Pwr Ctrl (MIO or EMIO)
- LED control, bus voltage (EMIO)
- Interrupt or polling driven
- AHB master-slave interface operating at the CPU_1x clock rate
- Master mode for DMA transfers (with 1 KB FIFO)
- Slave mode for register accesses

SDIO Specification 2.0

- Low-speed, 1 KHz to 400 KHz
- Full-speed, 1 MHz to 50 MHz (25 MB/sec)
- High-speed and high-capacity memory cards

2.2 ZynqMP/Versal

Compliance

- SD Host Controller Standard Specification Version 3.00
- SDIO card specification Version 3.0
- SD Memory Card Specification Version 3.01
- SD Memory Card Security Specification version 1.01
- MMC Specification version 4.51
- OCP specification version 2.01 (For the Host Controller with OCP Interface)
- AMBA AHB Specification version 2.00 (For the Host Controller with AHB Interface)
- AMBA AXI Specification version 3.00 (For Host Controller with AXI Interface)

System/Host Interface

- Supports one of the following System/Host Interfaces: AHB, AXI or OCP
- Data transfer using PIO mode on the Host Bus Slave interface, using DMA mode on the Host Bus Master interface. Here the Host Bus is AHB or AXI or OCP Interface.

SD/SDIO Card interface

- Host clock rate variable between 0 and 208 MHz
- Up to 832Mbits per second data rate using 4 parallel data lines (SDR104 mode)
- Transfers the data in 1 bit and 4 bit SD modes
- Transfers the data in SDR104, SDR50, DDR50 modes.
- Cyclic Redundancy Check CRC7 for command and CRC16 for data integrity

- Variable-length data transfers
- Performs Read wait Control, Suspend/Resume operation SDIO CARD.
- Designed to work with I/O cards, Read-only cards and Read/Write cards
- Supports Read wait Control, Suspend/Resume operation

MMC card interface

- Host clock rate variable between 0 and 208 MHz
- Up to 1664Mbits per second data rate using 8 bit parallel data lines (mmc8 bit SDR mode)
- Up to 832Mbits per second data rate using 8 bit parallel data lines (mmc8 bit DDR mode)
- Transfers the data in 1 bit, 4 bit and 8 bit modes
- Cyclic Redundancy Check CRC7 for command and CRC16 for data integrity
- Supports MMC Plus and MMC Mobile
- Card Detection (Insertion / Removal)

What's new in Zynqmp/Versal

1. SD
 - UHS speed modes
 - 1.8V capability
 - SDXC card capacity support (>64G)
 - Tuning procedure for SDR104/DDR50/SDR50
 - voltage switch, tuning commands
2. eMMC
 - complete new spec handled by JEDEC compared to MMC association
 - HS200 mode and Extended CSD register to support various features
 - 1.8V/1.2V support from CMD0
 - DDR mode support
 - 8-bit bus width
 - Tuning, bus width testing procedures
 - variants of erase - secure/trim/discard/sanitize
 - boot partitions, boot mode alternate boot mode
 - RPMB partitions

2.3 Features supported by driver

2.3.1 Zynq

- All the HW/IP features are supported by driver

2.3.2 ZynqMP

- All the HW/IP features are supported by driver

2.3.3 Versal

- All the HW/IP features are supported by driver

The two SDIO controllers are controlled and operate independently with the same feature set:

3 Missing features, known Issues, limitations

- SD UHS modes support is disabled in the driver currently due to board and silicon dependencies. Not all boards are having 3.0 level shifter

4 Kernel configurations

```
config MMC_SDHCI_OF_ARASAN
```

```
    tristate "SDHCI OF support for the Arasan SDHCI controllers"
```

```
    depends on MMC_SDHCI_PLTFM
```

```
    depends on OF
```

```
    help
```

```
        This selects the Arasan Secure Digital Host Controller Interface
        (SDHCI). This hardware is found e.g. in Xilinx' Zynq SoC.
        If you have a controller with this interface, say Y or M here.
        If unsure, say N.
```

```
<M> Sound card support --->
```

```
HID support --->
```

```
[*] USB support --->
```

```
<M> Ultra Wideband devices --->
```

```
<*> MMC/SD/SDIO card support --->
```

```
<M> Sony MemoryStick card support --->
```

```
-*- LED Support --->
```

```
--- MMC/SD/SDIO card support
```

```
[ ] MMC debugging
```

```
[ ] MMC host clock gating (NEW)
```

```
*** MMC/SD/SDIO Card Drivers ***
```

```
<M> MMC block device driver
```

```
(8) Number of minors per block device (NEW)
```

```
[*] Use bounce buffer for simple hosts
```

```
< > SDIO UART/GPS class support
```

```
< > MMC host test driver
```

```
*** MMC/SD/SDIO Host Controller Drivers ***
```

```
<*> Secure Digital Host Controller Interface support
```

```
< > SDHCI support on PCI bus
```

```
[*] Ricoh MMC Controller Disabler
```

```
<*> SDHCI platform and OF driver helper
```

```
<*> SDHCI OF support for the Arasan SDHCI controllers (NEW)
```

```
< > SDHCI support for Fujitsu Semiconductor F_SDH30 (NEW)
```

```
< > TI Flash Media MMC/SD Interface support
```

```
< > MMC/SD driver for Ricoh Bay1Controllers  
< > ENE CB710 MMC/SD Interface support  
< > VIA SD/MMC Card Reader Driver
```

5 Devicetree

SD and eMMC

```
sdhci@ff160000 {  
    compatible = "xlnx,zynqmp-8.9a", "arasan,sdhci-8.9a";  
    interrupt-parent = <&gic>;  
    interrupts = <0x0 0x30 0x4>;  
    reg = <0x0 0xff160000 0x0 0x1000>;  
    clock-names = "clk_xin", "clk_ahb";  
    xlnx,device_id = <0x0>;  
};
```

```
sdhci@ff170000 {  
    compatible = "xlnx,zynqmp-8.9a", "arasan,sdhci-8.9a";  
    interrupt-parent = <&gic>;  
    interrupts = <0x0 0x31 0x4>;  
    reg = <0x0 0xff170000 0x0 0x1000>;  
    clock-names = "clk_xin", "clk_ahb";  
    xlnx,device_id = <0x0>;  
};
```

6 Performance

Zynq:

High speed	20.54 MB/sec	read speed, tool: hdparm
-------------------	--------------	--------------------------

ZynqMP:

High Speed	19.4 MB/Sec	read speed, tool: hdparm
UHS (SDR)	SDR104: 76.50MB/sec	read speed, tool: hdparm
UHS (DDR)	DDR50: 40.68MB/sec	read speed, tool: hdparm

[click here](#)¹

SD card : Sandisk Ultra 16GB SDHC card
 Note : ZynqMP platform will support 'High Speed' mode by default.
 To operate the SD on UHS (Ultra High Speed) modes, please .

¹ <https://www.xilinx.com/support/answers/69978.html>

7 Test Procedure

Read/Write test using File System

```
mkfs.vfat -F 32 /dev/mmcblk0(p1)
mount /dev/mmcblk0(p1) /mnt
mkdir /mnt/sd
vi /mnt/sd/sd.txt
umount /mnt
```

Read/Write test using DD commands

```
dd if=/dev/urandom of=/tmp/data bs=1M count=10
dd if=/tmp/data of=/dev/mmcblk0(p1) bs=1M count=10
dd if=/dev/mmcblk0(p1) of=/tmp/data1 bs=1M count=10

md5sum /tmp/data /tmp/data1
(sha values reported by md5sum should be equal for data and data1 files)
```

Create partition using FDISK

```
List Partitions:
-----

fdisk -l /dev/mmcblk0

[ 1307.186442] mmc0: Got data interrupt 0x00000002 even though no data operation was
in progress.

Disk /dev/mmcblk0: 3947 MB, 3947888640 bytes

4 heads, 16 sectors/track, 120480 cylinders

Units = cylinders of 64 * 512 = 32768 bytes
```

Device	Boot	Start	End	Blocks	Id	System
/dev/mmcblk0p1		1	1	24	b	Win95 FAT32
/dev/mmcblk0p2		2	2	32	83	Linux

```
Create Partition:
-----

fdisk /dev/mmcblk0
```

```
[ 1344.587085] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

The number of cylinders **for this** disk is set to 120480.

There is nothing wrong with that, but **this** is larger than 1024, and could in certain setups cause problems with:

- 1) software that runs at boot time (e.g., old versions of LILO)
- 2) booting and partitioning software from other OSs
(e.g., DOS FDISK, OS/2 FDISK)

Command (m **for** help): n

Command action

e extended

p primary partition (1-4)

p

Partition number (1-4): 3

First cylinder (3-120480, **default** 3): 3

Last cylinder or +size or +sizeM or +sizeK (3-120480, **default** 120480): 3

Command (m **for** help): w

The partition table has been altered.

Calling ioctl() to re-read partition table

```
[ 1832.815341] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
[ 1832.833339] mmcblk0: p3
```

Different options:

Command (m **for** help): m

Command Action

a toggle a bootable flag

```

b      edit bsd disklabel
c      toggle the dos compatibility flag
d      delete a partition
l      list known partition types
n      add a new partition
o      create a new empty DOS partition table
p      print the partition table
q      quit without saving changes
s      create a new empty Sun disklabel
t      change a partition's system id
u      change display/entry units
v      verify the partition table
w      write table to disk and exit

```

Read/Write using File System:

```
root@Xilinx-ZynqMP-2015_3:~# mkfs.vfat -F 32 /dev/mmcblk0p3
```

```
[ 598.662380] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
[ 598.686123] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
root@Xilinx-ZynqMP-2015_3:~# mount /dev/mmcblk0p3 /mnt
```

```
[ 632.450944] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
[ 632.519516] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
[ 632.558846] mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
```

```
root@Xilinx-ZynqMP-2015_3:~# mkdir /mnt/sd
```

```
root@Xilinx-ZynqMP-2015_3:~# vi /mnt/sd/sd.txt
root@Xilinx-ZynqMP-2015_3:~# umount /mnt
```

Testing the eMMC RPMB Partition

The eMMC RPMB partition can be tested by using [mmc-utils](#)² tool.

Please note that writing the authentication key to the RPMB partition is one-time programmable (irreversible) change.

```
root@plnx_aarch64:~#
root@plnx_aarch64:~# vi key.txt
root@plnx_aarch64:~# cat key.txt
AAAABBBBCCCCDDDEEEFFFFFGGGGHHHH
root@plnx_aarch64:~#
root@plnx_aarch64:~# mmc rpmb write-key /dev/mmcblk0rpmb key.txt
root@plnx_aarch64:~#
root@plnx_aarch64:~# mmc rpmb read-counter /dev/mmcblk0rpmb
Counter value: 0x00000000
root@plnx_aarch64:~# vi temp1.txt
root@plnx_aarch64:~# ls -l temp1.txt
-rw-r--r--  1 root  users      256 Mar 28 12:51 temp1.txt
root@plnx_aarch64:~#
root@plnx_aarch64:~# cat temp1.txt
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
root@plnx_aarch64:~#
root@plnx_aarch64:~# mmc rpmb write-block /dev/mmcblk0rpmb 0x02 temp1.txt key.txt
root@plnx_aarch64:~#
root@plnx_aarch64:~# touch temp2.txt
root@plnx_aarch64:~# cat temp2.txt
root@plnx_aarch64:~#
root@plnx_aarch64:~# mmc rpmb read-block /dev/mmcblk0rpmb 0x02 1 temp2.txt key.txt
root@plnx_aarch64:~#
root@plnx_aarch64:~# cat temp2.txt
Have a good day
Have a good day
```

² <https://git.kernel.org/pub/scm/linux/kernel/git/cjb/mmc-utils.git/>

```

Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
Have a good day
root@plnx_aarch64:~#

```

7.1 Expected output

Boot log for SD

```

sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
sdhci-araman ff160000.sdhci: No vmmc regulator found
sdhci-araman ff160000.sdhci: No vqmmc regulator found
mmc0: SDHCI controller on ff160000.sdhci [ff160000.sdhci] using ADMA 64-bit
sdhci-araman ff170000.sdhci: No vmmc regulator found
sdhci-araman ff170000.sdhci: No vqmmc regulator found
mmc1: SDHCI controller on ff170000.sdhci [ff170000.sdhci] using ADMA 64-bit

.....

mmc0: new high speed SDHC card at address 1234
mmcblk0: mmc0:1234 SA04G 3.63 GiB
mmc0: Got data interrupt 0x00000002 even though no data operation was in progress.
mmcblk0: p1

.....

mmc1: new high speed SDHC card at address aaaa
mmcblk1: mmc1:aaaa SU08G 7.40 GiB
mmcblk1: p1

```

Boot log for EMMC

```

mmc0: BKOPS_EN bit is not set
mmc0: new high speed MMC card at address 0001

```

```
mmcblk0: mmc0:0001 MMC04G 3.57 GiB  
mmcblk0boot0: mmc0:0001 MMC04G partition 1 2.00 MiB  
mmcblk0boot1: mmc0:0001 MMC04G partition 2 2.00 MiB  
mmcblk0rpmb: mmc0:0001 MMC04G partition 3 128 KiB
```

```
mmcblk0:
```

```
mmcblk0boot1: unknown partition table  
mmcblk0boot0: unknown partition table
```

8 Mainline Status

In Sync with mainline driver.

9 Change Log

2024.2

Summary:

- Add Emmc hardware reset functionality

Commits:

[d8f66157ca6a](#)³ - Support for emmc hardware reset

2024.1

Summary:

- Change the return type of remove callback from int to void to make sure it will not do error handling.
- Use sdhci_pltfm_remove API instead of sdhci_pltfm_unregister API as this API is going to be deprecated.

Commits:

[1102e6f3f376](#)⁴ - Convert to platform remove callback returning void

[5d9acdf14358](#)⁵ - Use sdhci_pltfm_remove()

[7b101a81cdcd](#)⁶ - Explicitly include correct DT includes

2023.2

Summary:

- Sync Versal Net eMMC support with mainline related to eMMC5.1 compatible and code changes.

Commits:

[c9c461cf0f13](#)⁷ - Sync Versal Net eMMC support with mainline

2023.1

Summary:

- Add support for eMMC5.1 on Versal Net platform.
- Enable HS400 mode for Versal Net platform
- Add support to request the "gate" clock

Commits:

[8ebc7d2ab84b](#)⁸ - eMMC5.1 support added for Versal Net platform.

[0ab1396da6a9](#)⁹ - Enable HS400 mode support.

3 <https://github.com/Xilinx/linux-xlnx/commit/e2efd53c3dfa7bd84d32367ebbc0f585ca3bb97a>

4 <https://github.com/Xilinx/linux-xlnx/commit/ef1c3a7ec03b57e407bf3003095a5d9acdf14358>

5 <https://github.com/Xilinx/linux-xlnx/commit/3de205a07a70cb0a025adbb9797c1102e6f3f376>

6 <https://github.com/Xilinx/linux-xlnx/commit/c62da8a8a9872039ff3751d65a7f7b101a81cdcd>

7 <https://github.com/Xilinx/linux-xlnx/commit/c9c461cf0f134d822347fa2b7c674a125ec0ea3d>

8 <https://github.com/Xilinx/linux-xlnx/commit/8ebc7d2ab84bc7fbc05bf222db894ac84fb2c26a>

9 <https://github.com/Xilinx/linux-xlnx/commit/0ab1396da6a9d086bba56df7f3786d2413244669>

[ad43515d14a8](#)¹⁰ - Add support to request the gate clock.

2022.2

Summary:

- Add NULL check for data field in probe.

Commits:

[b442098dd64c](#)¹¹ - Add NULL check for data field.

2021.2

Summary:

- None

2022.1

Summary:

- Add support for dynamic configuration
- Add 1msec delay after controller reset to reach eMMC card stable state

Commits:

[3c7719dbfbdf](#)¹² - Add support for dynamic configuration for SOM.

[0349189baeca](#)¹³ - Add delay after controller reset to reach eMMC card stable state

2021.2

- None

2021.1

Summary:

- Fix the issue in reading the tap delay values from Devicetree.

Commits:

[1007e369c036](#)¹⁴ - Fix the issue in reading tap values from DT

¹⁰ <https://github.com/Xilinx/linux-xlnx/commit/ad43515d14a88666f732f9cf6eea09c72931b259>

¹¹ <https://github.com/Xilinx/linux-xlnx/commit/b442098dd64cbad073dcc0e24da54403248e3d32>

¹² <https://github.com/Xilinx/linux-xlnx/commit/3c7719dbfbdfa8fed6089b4974ec7edf55a7ade4>

¹³ <https://github.com/Xilinx/linux-xlnx/commit/0349189baecaf90d8a44dafd765bbaa8b63b9098>

¹⁴ <https://github.com/Xilinx/linux-xlnx/commit/1007e369c0362c5fcc00cf5d86f9afbc0488c789#diff-a3bcbcc128093b2b25d802e467ee9b9dc1a5def2df07b0da03f4a0c6c624850b>

10 Related Links

Source file link:

<https://github.com/Xilinx/linux-xlnx/blob/master/drivers/mmc/host/sdhci-of-arsan.c>